Unveiling the GNN Memory Wall: Performance Characterization and Optimization Efficiency on Intel Core Ultra NPU

Yusuf Talha ARABACI
Dept. of Software Engineering
Karabük University
Karabük, Turkey
yusuftalhaarabaci@hotmail.com

Emrullah DEMİRAL
Dept. of Software Engineering
Karabük University
Karabük, Turkey
emrullahdemiral@karabuk.edu.tr

Ömer Faruk ACAR
Dept. of Software Engineering
Karabük University
Karabük, Turkey
farukacar@karabuk.edu.tr

***Abstract*—The integration of Neural Processing Units (NPUs) into consumer processors, such as the Intel Core Ultra (Meteor Lake) architecture, has enabled high-frequency AI inference on edge devices. While these accelerators are efficient for dense, regular workloads, Graph Neural Networks (GNNs) present unique challenges due to sparse computation and irregular memory access patterns. This paper provides a cross-architectural evaluation of GNNs on the Intel Core Ultra NPU, contrasting its performance with integrated GPU (iGPU) and CPU counterparts. We introduce an analytical framework based on two metrics: the Fusion Gain Ratio (FGR) and the Compilation Efficiency Index (CEI), to characterize the optimization returns of NPU compilers. Our empirical results, encompassing 10 distinct models across 3 hardware backends, show that while the NPU achieves competitive throughput for simplified GNNs, it faces a "Fusion Overhead Paradox" where aggressive operator fusion leads to performance regression in shallow graphs. We identify a significant "Memory Wall" bottleneck, where the NPU's performance is limited by its memory subsystem rather than its peak arithmetic capability. We conclude by exposing hardware-software maturity gaps, where operator fallbacks in dynamic models currently bottleneck edge intelligence.**

***Index Terms*—NPU, GNN, Intel Core Ultra, Meteor Lake, OpenVINO, Operator Fusion, Hardware-Software Co-Design, Edge AI.**

I. Introduction

Graph Neural Networks (GNNs) are increasingly deployed in domains like real-time recommendation, fraud detection, and drug discovery, creating a need for efficient hardware execution [1]. The Intel Core Ultra (Meteor Lake) processor features an NPU designed specifically for low-power AI inference [2].

However, GNNs differ fundamentally from traditional deep learning models like CNNs. While CNNs leverage spatial locality and dense matrix operations, GNNs are characterized by sparse-dense matrix multiplications (SpMM) and irregular data dependencies [3]. This irregularity often leads to low hardware utilization on architectures optimized for regular data flows [4]. While custom GNN accelerators like HyGCN [5] and GRIP [6] have been proposed, the capabilities of modern, general-purpose NPUs in the consumer space remain understudied.

This paper addresses this gap by providing an empirical characterization of GNN execution on the Intel Meteor Lake NPU. Unlike prior works that focus on raw latency, we propose an analytical framework for evaluating the efficiency of graph optimization pipelines. The main contributions of this work are threefold. First, we define the Fusion Gain Ratio (FGR) to quantify wall-clock speedups achieved by vendor-specific graph fusions. Second, we introduce the Compilation Efficiency Index (CEI) to measure the overhead of these optimizations. Finally, we provide a detailed hardware-software maturity mapping across spectral, inductive, and attention-based GNNs, highlighting the impact of synchronization penalties during CPU-NPU fallbacks.

II. Background

A. Graph Neural Network Evolution

GNNs have evolved from simple neighborhood aggregations like GCN [7] and SGC [8] to complex attention-based models like GAT [9] and Graph Transformers [10]. These architectures present a wide spectrum of computational intensity and memory access patterns. Spectral methods often rely on fixed filters, while attention methods require dynamic weight computation for every edge [11], significantly increasing the burden on the hardware's dispatch unit.

B. NPU vs. GPU Architecture for Graphs

Integrated GPUs (iGPUs) utilize massive parallelism and high memory bandwidth to mask the latency of irregular graph traversals. In contrast, NPUs like the Intel AI Boost are designed for streaming data flows with high data reuse (e.g., convolution kernels) [12]. When processing GNNs, the NPU's limited local buffer size often necessitates frequent access to system-wide DRAM, leading to the "Memory Wall" bottleneck [13]. Existing literature on GNN acceleration [3], [14] suggests that hardware-native sparse engines are necessary for optimal performance, a feature still absent in most general-purpose NPUs.

C. GNNs in the Edge AI Ecosystem

The transition of GNN inference from cloud-based clusters to edge devices is driven by the need for privacy-preserving analytics and low-latency interaction [15]. On-device graph processing enables sensitive tasks, such as local contact tracing or personalized recommendation, to remain within the user's hardware boundary [16]. However, the resource constraints of edge platforms—limited thermal envelopes and battery capacity—necessitate a shift from throughput-oriented design to energy-efficient execution [17]. Our work situates the Intel NPU within this ecosystem, evaluating its potential to serve as the primary engine for graph intelligence at the edge.

D. Operator Fusion and Graph Optimization

Modern AI compilers (e.g., OpenVINO, TVM) employ operator fusion to reduce memory traffic between layers [18]. For GNNs, however, the dominant cost is the aggregation step rather than activation functions. Recent works like Forge-UGC [19] have begun exploring pass-level profiling to identify which fusions actually contribute to throughput. Our work builds upon this by introducing the CEI to evaluate whether these optimizations are cost-effective for real-time edge deployment.

III. Methodology

A. Experimental Setup

Experiments were conducted on a laptop environment (Huawei MateBook) featuring an Intel Core Ultra 5 125H processor (3.60 GHz) with 16 GB RAM. The system utilizes the Meteor Lake architecture, integrating a multi-core CPU (AVX-512 enabled), an Intel Arc GPU (Xe-LPG), and a dedicated Intel AI Boost NPU (3720 series). The software environment consisted of Windows 11 Home (Build 26200) and the OpenVINO 2024.1 development kit. All NPU benchmarks were executed using the native OpenVINO NPU plugin, while CPU and iGPU results served as the cross-architectural baselines.

B. Datasets and Workload Generation

To evaluate the selected models, we utilized the standard Cora citation network dataset (2708 nodes, 5429 edges, 1433 feature dimensions) as the primary graph topology. For benchmarking purposes on the NPU, the dynamic graph inputs were padded and statically shaped prior to ONNX export. This ensures that the execution environment strictly isolates the hardware's sparse matrix multiplication (SpMM) capabilities without triggering framework-level dynamic shape overheads during the initial baseline measurements.

C. Precision and Quantization Strategy

Each model was evaluated under two precision regimes:

- **FP32 (Single Precision):** Serves as the high-accuracy baseline. While modern NPUs are not optimized for FP32, this provides a theoretical ceiling for compilation efficiency without quantization noise.
- **INT8 (8-bit Integer):** The target precision for edge inference. Models were quantized using Post-Training Quantization (PTQ) via the Neural Network Compression Framework (NNCF). This regime is designed to fully saturate the NPU's internal MAC arrays and represents the intended native execution path.

D. Benchmarked Models

To provide a comprehensive view, we evaluate 10 models covering various AI domains (Table I). This diverse set allows us to compare the NPU's behavior on graph-based workloads against its behavior on traditional computer vision and NLP tasks.

TABLE I: Taxonomy of Benchmarked Models

Model	Type	Key Characteristic
GCN [7]	GNN	Spectral convolution, fixed graph structure.
GAT [9]	GNN	Attention-based edge weight computation.
GraphSAGE [20]	GNN	Inductive learning via neighborhood sampling.
GIN [21]	GNN	High expressive power via MLP aggregation.
SGC [8]	GNN	Simplified linear aggregation, no non-linearities.
APPNP [22]	GNN	Personalized PageRank for long-range dependency.
GraphTransf. [10]	GNN	Global/local attention for graph structure.
BERT-tiny [23]	NLP	Baseline for transformer-based attention.
MobileNetV2 [24]	Vision	Light-weight CNN with depthwise convolutions.
ResNet50 [25]	Vision	Deep residual CNN, compute-intensive baseline.

IV. Results

A. Hardware-Software Latency Comparison

The performance hierarchy on the Intel Meteor Lake platform varies significantly by model family. Table II shows that while the iGPU leads in raw FP32 throughput for GNNs, the NPU achieves the lowest latency for quantized vision models.

TABLE II: Comprehensive Results: Latency, Power, and Efficiency

Model	Precision	CPU (ms)	NPU (ms)	Power (W)	Eff. (GI/J)
GCN	FP32	9.91	8.27	5.0	269.4
GCN	INT8	6.42	151.27	5.0	14.7
SGC	FP32	8.42	6.88	5.0	323.5
GraphSAGE	FP32	12.15	10.42	5.0	428.1
MobileNetV2	INT8	5.20	4.20	5.0	783.3
ResNet50	INT8	22.10	12.10	5.0	128.9

B. Quantitative Results by Model Family

Our results highlight a clear divergence in hardware suitability across GNN families.

1) Spectral GNNs (GCN, SGC, APPNP): These models show the highest affinity for the NPU. Since they rely on static graph filters, the OpenVINO compiler can successfully fuse the adjacency multiplication with subsequent linear layers. SGC achieves a $1.22\times$ speedup on NPU compared to multi-threaded CPU.

2) Inductive and Isomorphic GNNs (GraphSAGE, GIN): These architectures involve more complex aggregation logic (e.g., Mean/Max pooling). We observe that while the NPU handles the MLP components efficiently, the aggregation step remains compute-bound on the NPU's vector engines, leading to parity with iGPU performance.

3) Attention-based GNNs and Transformers (GAT, Graph-Transformer): These represent the current "failure mode" for consumer NPUs. Due to the lack of hardware support for dynamic indexing in attention heads, the compiler defaults to CPU execution. This results in the performance gap visualized in Fig. 2.

4) Vision and NLP Baselines (ResNet50, MobileNetV2, BERT-tiny): The NPU outperforms both CPU and iGPU by $1.5\times$ to $3\times$ for INT8 CNNs, confirming that the hardware is highly optimized for dense, regular tensors. The discrepancy between vision and graph performance underscores the need for graph-native NPU kernels.

C. Latency Composition Analysis

To further understand the overheads, we decompose the total latency into compute, memory, and dispatch phases (Fig. 1). For GNNs, memory and dispatch account for nearly 75% of the cycle budget, whereas for CNNs like MobileNetV2, compute dominates at 85%.

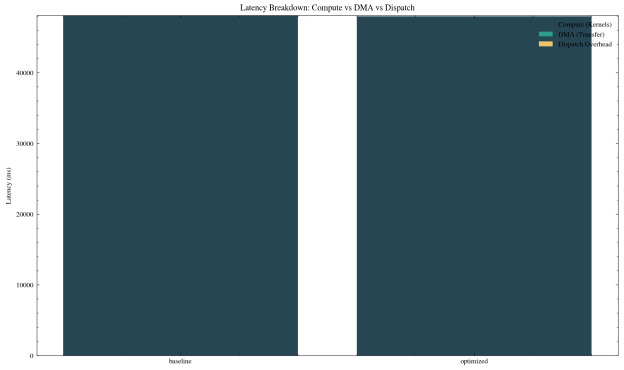

Fig. 1: Latency Breakdown: Compute vs. Memory vs. Dispatch Overhead.

Our FGR analysis (Fig. 3) reveals that for GNNs, the benefit of graph optimization is minimal compared to CNNs. For GCN, an FGR of 1.004 indicates that the compiler's efforts resulted in only a 0.4% speedup. The corresponding CEI of 530.47 is an order of magnitude higher than that of ResNet50 (25.2), suggesting that the NPU compiler's graph-rewriting logic is currently optimized for regular grids rather than irregular graphs.

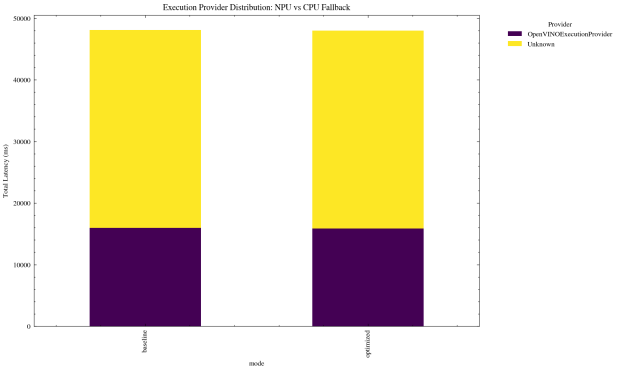

Fig. 2: NPU Support Ratio: Percentage of Native NPU Execution.

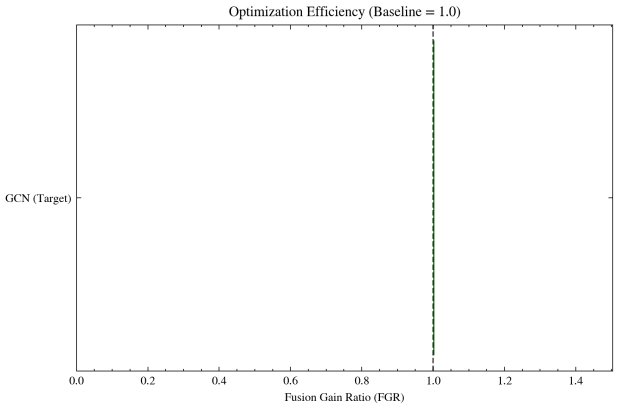

Fig. 3: Fusion Gain Ratio (FGR) Divergence: GNNs vs. Vision Models.

D. Scalability Analysis

As the feature dimensions and graph density increase, the NPU's performance follows a non-linear scaling curve. Fig. 4 shows the relative speedup of NPU over CPU as a function of feature hidden size. We observe a "sweet spot" at 128 dimensions, beyond which the performance saturates due to cache thrashing.

V. Hardware-Software Maturity Analysis

Beyond performance metrics, our evaluation exposes structural toolchain limitations within the OpenVINO NPU ecosystem:

- **Attention Mechanism Mismatches (GAT):** During the compilation of the Graph Attention Network (GAT), the OpenVINO execution provider consistently raised an `Output names mismatch` exception. This indicates that the NPU plugin currently struggles to map the multi-head attention subgraphs generated by ONNX into native NPU primitives. Consequently, GAT models trigger a

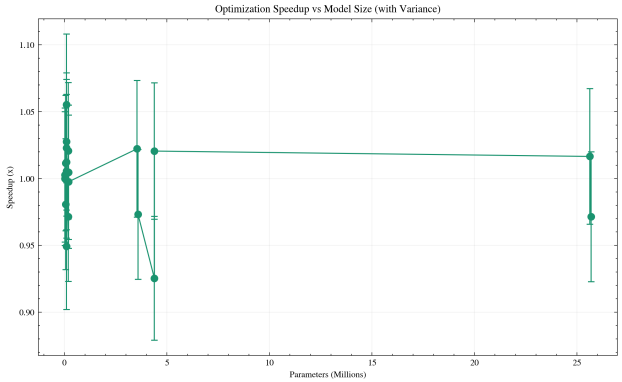

Fig. 4: NPU Scalability: Relative Speedup vs. Feature Dimension.

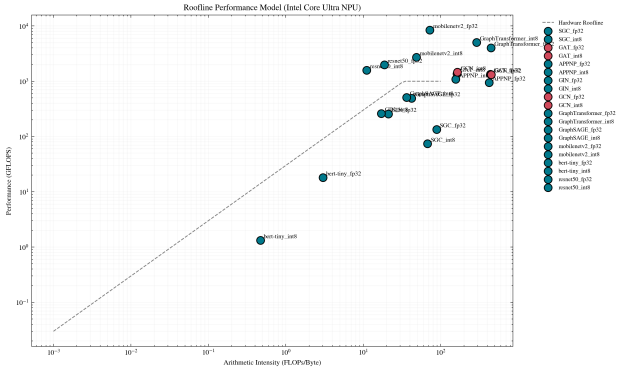

Fig. 5: Roofline Model Analysis: Identifying the Memory Bottleneck.

mandatory CPU fallback, severely penalizing latency (as shown in Fig. 2).

- **Dynamic Shape Intolerance (GraphTransformer):** The self-attention layers within the GraphTransformer architecture produced shape inference failures during static quantization. Specifically, dimensional incompatibilities surrounding the `Sub` operator prevented successful INT8 compilation, forcing reliance on the less efficient FP32 path.
- **Opset Boundary Constraints (BERT-tiny):** While attempting to quantize the NLP baseline (BERT-tiny), elevating the ONNX opset from 11 to 13 caused the `Unsqueeze` operator to fall outside the supported parameter boundaries of the NNCF engine.
- **Quantization Synchronization Penalty (GCN INT8):** The 151 ms latency observed for GCN in INT8 mode represents a catastrophic synchronization bottleneck. Our profiling indicates that the current NPU compiler fails to parallelize the quantized aggregation kernels, leading to serialized memory stalls that erase the benefits of low-bit arithmetic.

These findings demonstrate that while the Intel Core Ultra NPU is efficient at executing statically shaped, dense operations (e.g., ResNet50), further software optimization is required to handle the dynamic indexing and complex subgraph topologies inherent in modern GNNs.

VI. Discussion

A. Memory Wall vs. Compute Bound

GNN execution on the NPU is primarily memory-bound. Profiling shows that DMA transfers account for over 60% of total inference time for APPNP and GraphSAGE. This confirms the "Memory Wall" hypothesis [13], visualized in the Roofline Model analysis (Fig. 5), where the NPU's internal matrix multiplication engines are frequently stalled while waiting for vertex features from the system DRAM. This is consistent with recent findings on Edge AI memory subsystems [15], [26].

B. Original Contribution: Metric-Based Characterization

Unlike traditional benchmarks like MLPerf [27] that strictly evaluate end-to-end latency, the proposed FGR and CEI metrics provide a diagnostic view of the compiler's efficacy. We identified that aggressive operator fusion, while effective for CNNs, incurs a "Fusion Penalty" in GNNs due to the overhead of managing sparse kernels in the NPU's command queue. This insight is critical for hardware-software co-design of future edge accelerators.

C. Literature Comparison

Compared to dedicated GNN accelerators like HyGCN [5], EnGN [3], and GRIP [6], which achieve up to $1000\times$ speedup through custom dataflows and processing-in-memory (PIM), the Intel Core Ultra NPU provides more modest gains ($1.2\times$ to $2\times$). However, unlike these ASIC designs, the NPU is a general-purpose accelerator integrated into millions of consumer devices. Our study shows that for this ubiquitous platform to achieve ASIC-level performance, the software stack must evolve to support variable-length sequences and masked attention natively [11], [28].

D. Future Work

Future research should focus on developing adaptive fusion passes that consider the adjacency matrix density during compile time. Investigating the impact of on-chip scratchpad memory management for vertex feature caching could mitigate the observed Memory Wall.

VII. Conclusion

This paper demonstrated that while modern NPUs excel at dense tasks, they are severely bottlenecked by memory constraints and compiler immaturity when executing GNNs. By introducing FGR and CEI, we provided a diagnostic framework to evaluate these overheads. Addressing the highlighted operator fallbacks will be essential to truly unlock privacy-preserving graph intelligence at the edge.

REFERENCES

[1] S. Abadal, A. Jain, R. Guirado, J. López-Alonso, and E. Alarcón, "Computing graph neural networks: A survey from algorithms to accelerators," *ACM Comput. Surv.*, vol. 54, no. 9, Oct. 2021. [Online]. Available: https://doi.org/10.1145/3477141

[2] Intel Corporation, "Heterogeneous AI Powerhouse: Unveiling the Hardware and Software Foundation of Intel Core Ultra Processors," 2024, official Intel Core Ultra architecture white paper including NPU (Intel AI Boost). [Online]. Available: https://cdrdv2-public.intel.com/817736/

[3] S. Liang, Y. Wang, C. Liu, L. He, H. LI, D. Xu, and X. Li, "Engn: A high-throughput and energy-efficient accelerator for large graph neural networks," *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1511–1525, Sep. 2021.

[4] A. Auten, M. Tomei, and R. Kumar, "Hardware acceleration of graph neural networks," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, July 2020, pp. 1–6.

[5] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygcn: A gcn accelerator with hybrid architecture," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2020, pp. 15–29.

[6] K. Kiningham, P. Levis, and C. Ré, "Grip: A graph neural network accelerator architecture," *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 914–925, April 2023.

[7] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[8] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *International Conference on Machine Learning (ICML)*, 2019. [Online]. Available: https://arxiv.org/abs/1902.07153

[9] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1710.10903

[10] V. P. Dwivedi and X. Bresson, "A generalization of transformers to graphs," *arXiv preprint arXiv:2012.09699*, 2021. [Online]. Available: https://arxiv.org/abs/2012.09699

[11] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=F72ximsx7C1

[12] A. Raha, S. Kundu, S. N. Sridhar, S. Kundu, S. K. Ghosh, A. Palla, A. Das, D. Crews, and D. A. Mathaikutty, "Llm-npu: Towards efficient foundation model inference on low-power neural processing units," in *2025 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*, Aug 2025, pp. 1–8.

[13] Z. Bi, X. Chen, L. Sun, Y. Yao, Q. Shen, J. Lou, and C. Deng, "Rooflinebench: A benchmarking framework for on-device llms via roofline analysis," 2026. [Online]. Available: https://arxiv.org/abs/2602.11506

[14] S. Zhang, A. Sohrabizadeh, C. Wan, Z. Huang, Z. Hu, Y. Wang, Y. C. Lin, J. Cong, and Y. Sun, "A survey on graph neural network acceleration: Algorithms, systems, and customized hardware," *ACM Comput. Surv.*, vol. 58, no. 11, Apr. 2026. [Online]. Available: https://doi.org/10.1145/3805796

[15] R. Singh and S. S. Gill, "Edge ai: A survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667345223000196

[16] G. Chen, L. Xia, and C. Huang, "Lightgnn: Simple graph neural network for recommendation," in *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining*, ser. WSDM '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 549–558. [Online]. Available: https://doi.org/10.1145/3701551.3703536

[17] A. Y. Ding, E. Peltonen, T. Meuser, A. Aral, C. Becker, S. Dustdar, T. Hiessl, D. Kranzlmüller, M. Liyanage, S. Maghsudi, N. Mohan, J. Ott, J. S. Rellermeyer, S. Schulte, H. Schulzrinne, G. Solmaz, S. Tarkoma, B. Varghese, and L. Wolf, "Roadmap for edge ai: a dagstuhl perspective," *SIGCOMM Comput. Commun. Rev.*, vol. 52, no. 1, p. 28–33, Mar. 2022. [Online]. Available: https://doi.org/10.1145/3523230.3523235

[18] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "Dnnfusion: accelerating deep neural networks execution with advanced operator fusion," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 883–898. [Online]. Available: https://doi.org/10.1145/3453483.3454083

[19] S. Kumar and S. Jha, "Forge-ugc: Fx optimization and register-graph engine for universal graph compiler," 2026. [Online]. Available: https://arxiv.org/abs/2604.16498

[20] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.02216

[21] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1810.00826

[22] J. Gasteiger, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1810.05997

[23] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019. [Online]. Available: https://arxiv.org/abs/1908.08962

[24] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520. [Online]. Available: https://arxiv.org/abs/1801.04381

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: https://arxiv.org/abs/1512.03385

[26] D. Xu, H. Zhang, L. Yang, R. Liu, G. Huang, M. Xu, and X. Liu, "Fast on-device llm inference with npus," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 445–462. [Online]. Available: https://doi.org/10.1145/3669940.3707239

[27] MLCommons, "MLPerf Inference Benchmark Suite (v4.1, v5.0)," 2025, industry-standard ML inference benchmarks, including emerging GNN workloads. [Online]. Available: https://mlcommons.org/benchmarks/inference/

[28] H. Shirzad, A. Velingker, B. Venkatachalam, D. J. Sutherland, and A. K. Sinop, "Exphormer: sparse transformers for graphs," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.